

TEDEX Technical Manual
A Chaptered Guide to Thermoelectric Data Analysis

Northwestern Thermoelectrics Lab

June 23, 2026

Contents

Manual Outline	1
1 Introduction	2
1.1 Purpose	2
1.2 Planned Content	2
2 Workspace Architecture	3
2.1 Repository Layout	3
2.2 Planned Content	3
3 Raw TE Analysis	4
3.1 Workflow Goal	4
3.2 Command	4
3.3 Analysis Output	4
3.4 Noitons	5
4 SPB Fitting	6
4.1 Objective	6
4.2 SPB Model Used in the Code	6
4.3 Inputs and Outputs	7
4.4 Effective-Mass Fitting	7
4.5 Performance Fitting	8
4.6 Conductivity-Axis Fitting	10
4.7 Interpreting the Outputs	10
4.8 Quality-Control Checklist	12
4.9 Open Slots for Future Expansion	12
5 Plotting and Figures	13
5.1 Introduction	13
5.2 Fast TE Plotting	13
5.2.1 TE properties comparison inter batches	14
5.3 Planned Content	14
6 XRD Analysis	15
6.1 Scope	15
6.2 Command Slot	15
6.3 Planned Content	15
7 Assessment and Prediction	16
7.1 Scope	16
7.2 Command Slot	16
7.3 Planned Content	16

8	Reproducible Operations	17
8.1	End-to-End Protocol	17
8.2	Command Slot	17
8.3	Output Log Slot	17

Manual Outline

This manual is organized as a chaptered technical document. The goal is to make the project explainable, reproducible, and easy to extend chapter by chapter.

Table 1: Planned chapter structure.

No.	Chapter	Scope
1	Introduction	Research motivation, workspace goals, intended users, and the problem TEDEX solves.
2	Workspace Architecture	Folder layout, source-code boundaries, data ownership, and generated-output conventions.
3	Raw TE Analysis	ZEM/LFA ingestion, metadata requirements, transport property calculation, and extracted feature files.
4	SPB Fitting	Effective-mass fitting, performance curves, conductivity-axis fitting, command templates, outputs, and interpretation.
5	Plotting and Figures	Standard TE figures, flexible plotting recipes, style conventions, and publication-ready export formats.
6	XRD Analysis	XRD data layout, normalized/stacked plots, PDF-card overlays, and lattice-parameter fitting.
7	Assessment and Prediction	Batch assessment, AI-assisted interpretation, Bayesian prediction, and follow-up candidate ranking.
8	Reproducible Operations	End-to-end batch protocol, logging, figure slots, and recommended archival practice.

Editing Convention

Each chapter should include three practical components:

1. a conceptual description of the workflow;
2. command slots with exact bash commands;
3. terminal-output and figure slots that can be filled after each real run.

Chapter 1

Introduction

1.1 Purpose

TEDEX is a structured workspace for thermoelectric materials analysis. It is designed to connect raw laboratory measurements, sample metadata, Python analysis scripts, generated figures, and written reports without forcing every experiment into a rigid database-first workflow.

1.2 Planned Content

- Scientific motivation: why reproducible TE analysis matters for composition screening and optimization.
- Target users: lab members processing CHY batches, preparing figures, or comparing measured trends against SPB models.
- Design principles: runnable root scripts, reusable source modules, explicit metadata ledgers, and separated generated outputs.
- Short comparison to notebook-only workflows.

Drafting slot. Add one paragraph describing the specific material systems and experimental questions this manual should emphasize.

Chapter 2

Workspace Architecture

2.1 Repository Layout

The workspace keeps root-level scripts as user-facing entry points and stores implementation details in importable modules under `src/`. Source data, processed data, reports, and figures are separated to make outputs easy to inspect and clean.

Table 2.1: Core workspace directories.

Path	Role
<code>src/</code>	Reusable Python modules.
<code>src/tools/</code>	Data parsing, TE plotting, XRD utilities, and SPB fitting.
<code>data/raw/</code>	Original instrument files and raw batch folders.
<code>data/processed/</code>	Cleaned transport-property CSV tables.
<code>data/lab/</code>	Batch/sample ledgers and editable lab metadata.
<code>results/</code>	Reports, assessment payloads, fitting outputs, predictions.
<code>outputs/</code>	Figures, one-off exports, tables, and logs.

2.2 Planned Content

- Data ownership rules for raw, processed, and generated files.
- Naming conventions for CHY batches and samples.
- Relationship between `results/` and `outputs/`.
- How external snapshots under `external/` should be treated.

Chapter 3

Raw TE Analysis

3.1 Workflow Goal

The raw TE workflow converts paired ZEM/LFA measurements into processed transport-property tables. The standard entry point is `python main.py analyze`.

3.2 Command

The first step is to run a scrip (`scripts/sync_lab_metadata.py`) to sync sample information into a notebook (`data/lab/lab_metadata.md`), which record detailed information of experiemnts samples (batch id, comptosition, density, heat capacity, doping elements, etc.). This script will automatically detect the new added samples in `\data \raw` , and add new template in markdown. The *density* and *heat capacity* must be filled before running TE Analysis program.

Listing 3.1: Sync batch information.

```
# TODO: sync batch/sample information to \code{data/lab/lab_metadata.md}.
python scripts/sync_lab_metadata.py
```

Listing 3.2: Raw TE analysis command slot.

```
# TODO: replace with the exact batch or sample selector.
python main.py analyze CHY-1048

# Optional selected-sample analysis.
python main.py analyze CHY-1054 --sample B C
```

3.3 Analysis Output

Listing 3.3: Raw TE analysis terminal output slot.

```
# TODO: paste the key processing lines here.
# Example:
# step1. Data processing
# Saved processed data to: data/processed/...
# Saved extracted features to: results/...
```

Processed data will be located at `data/processed`, with extracted raw data, and lattice thermal conductivity (κ_L), quality factor based on SPB model, and ZT value.

3.4 Noitons

- Required ZEM/LFA file naming and raw folder structure.
- Required metadata: density, cp_value, sample ID, composition.
- Output CSV columns and extracted feature JSON.
- Common failure modes: missing LFA/ZEM files, missing density, unit mismatch, malformed instrument export.

Chapter 4

SPB Fitting

4.1 Objective

The single parabolic band (SPB) fitting tools provide a bridge between measured transport data and interpretable transport parameters. In TEDEX, the SPB workflow is not one monolithic command. It is split into three complementary entry points:

1. `effective_mass_fit.py`: fit density-of-states effective mass from Hall carrier concentration and Seebeck data.
2. `performance_fit.py`: use Hall concentration, Seebeck, power factor, and optionally zT to model power-factor and zT trends.
3. `conductivity_fit.py`: fit weighted mobility on a conductivity axis and plot Seebeck, power factor, and optionally zT versus conductivity.

The practical goal is to answer three questions: what effective mass is consistent with the measured Pisarenko trend, which carrier concentration or conductivity regime maximizes performance, and whether the measured data sit on a physically plausible SPB curve.

4.2 SPB Model Used in the Code

The current implementation uses a compact acoustic-phonon SPB model. The reduced Fermi level η is obtained from the measured Seebeck magnitude. For the default scattering exponent, the Seebeck magnitude is evaluated as

$$|S| = \frac{k_B}{e} \left(\frac{2F_1(\eta)}{F_0(\eta)} - \eta \right), \quad (4.1)$$

where $F_j(\eta)$ denotes the unnormalized Fermi integral

$$F_j(\eta) = \int_0^\infty \frac{x^j}{1 + \exp(x - \eta)} dx. \quad (4.2)$$

For a density-of-states effective mass m^* , the true carrier concentration is computed as

$$n = 4\pi \left(\frac{2m^* m_e k_B T}{h^2} \right)^{3/2} F_{1/2}(\eta). \quad (4.3)$$

By default, the Hall concentration is modeled as

$$n_H = \frac{n}{r_H}, \quad (4.4)$$

where r_H is the acoustic-SPB Hall factor. The command-line option `--no-hall-factor` switches to the simplified convention $n_H = n$.

4.3 Inputs and Outputs

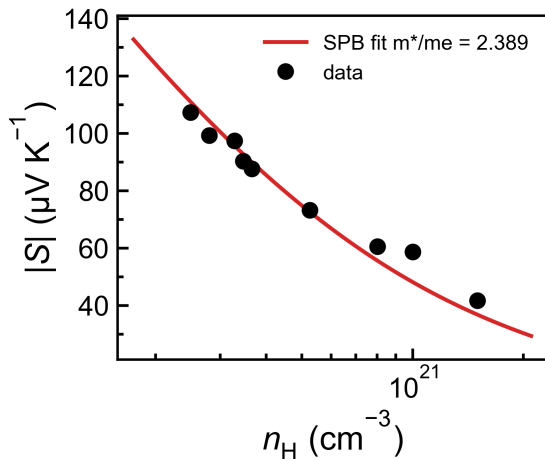
Table 4.1: SPB fitting modes and their expected files.

Mode	Required data	Main outputs
Effective mass	n_H , S ; optional <code>curve_id</code> , T	<code>fit_summary.json</code> , <code>effective_mass_points.csv</code> , <code>pisarenko_curve.csv</code> , <code>pisarenko_fit.png/pdf</code>
Performance	n_H , S , PF ; optional ZT , parameters	<code>performance_summary.json</code> , <code>performance_points.csv</code> , <code>performance_curve.csv</code> , <code>pf_fit.png/pdf</code> , <code>zt_fit.png/pdf</code>
Conductivity axis	conductivity, S ; optional PF , ZT , kL	<code>conductivity_summary.json</code> , <code>conductivity_points.csv</code> , <code>conductivity_curve.csv</code> , <code>conductivity_*_fit.png/pdf</code>

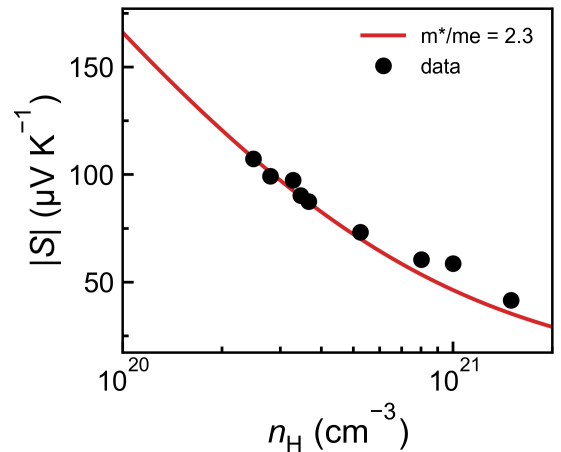
4.4 Effective-Mass Fitting

This mode is the first pass for a Hall-Pisarenko analysis. The script reads a CSV containing Hall concentration and Seebeck columns, solves η from Seebeck, and fits the effective mass that best maps the SPB carrier concentration curve to the measured n_H values.

Single-sample fitting will automatically calculate average density-of-states effective mass based on provided data. We can manually define the effective mass by `--m 2.3` and alter fitting range by `--xlim 1e20 2e21`.



(a) Automatic effective-mass fitting result.

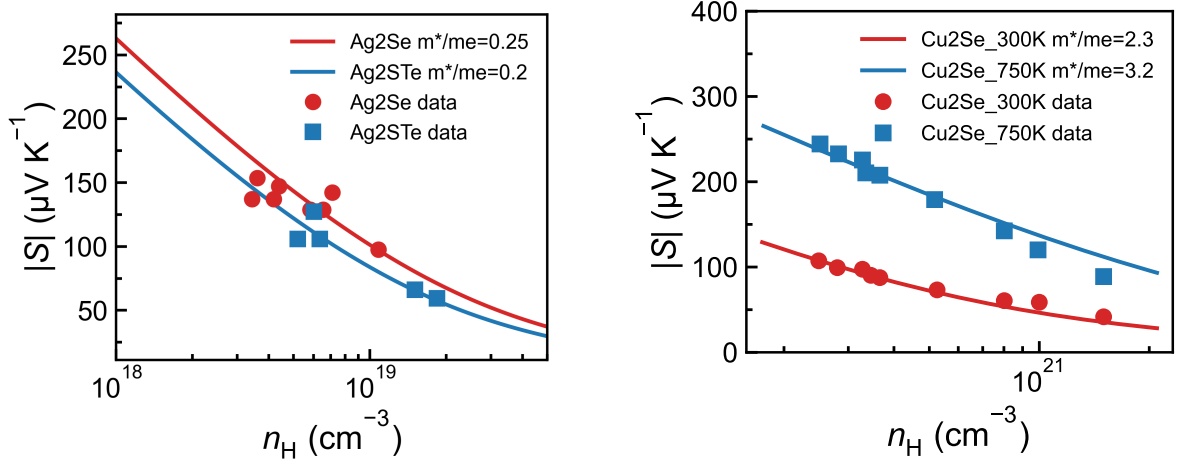


(b) Manual effective-mass fitting result.

Figure 4.1: Pisarenko fitting curves for Cu_2Se at 300 K.

When multiple samples are required for fitting, we can use the `--group-by curve_id` option to group the samples by `curve_id`. This requires an original column named `curve_id` to be

present in the input data. The paramter CSV is also need to identify the fitting temperature if not at default 300 K.



(a) Fitting of Ag2Se and Ag2SeTe at 300 K.

(b) Fitting of Cu2Se at 300 K and 750 K.

Figure 4.2: Multi-groupPisarenko fitting curves.

Listing 4.1: Command slot: SPB effective-mass fit.

```
python main.py spb effective-mass \
  data/demo/spb_fitting/Ag2Se/Ag2Se_series.csv \
  --group-by curve_id \
  --params data/demo/spb_fitting/Ag2Se/Ag2Se_params.csv \
  --output-dir results/spb_fitting/effective_mass_fit \
  --formats png pdf \
  --legend outside \
  --no-show
```

Listing 4.2: Terminal-output slot: effective-mass fit.

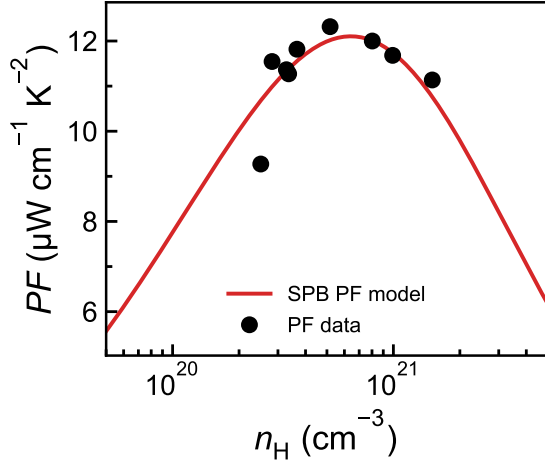
```
# TODO: paste key saved-file lines here.
# Expected outputs include:
# results/spb_fitting/effective_mass_fit/fit_summary.json
# results/spb_fitting/effective_mass_fit/effective_mass_points.csv
# results/spb_fitting/effective_mass_fit/pisarenko_curve.csv
# results/spb_fitting/effective_mass_fit/pisarenko_fit.png
```

4.5 Performance Fitting

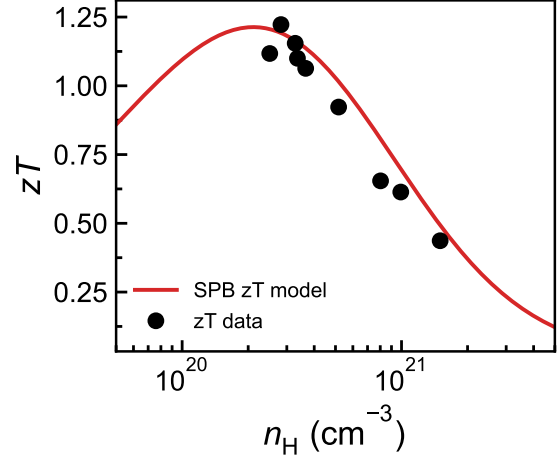
The performance fit extends the Pisarenko analysis by fitting or using a mobility model and then rendering power-factor and zT curves. This is useful when the question is not only “what m^* explains $Sv.s.(n_H)$?” but also “where should performance peak as carrier concentration changes?”

The performance fitting also supports single data, which just requires input fitting paramters in CLI line or by the computer. If multiple curves is necessary, it is recommaned to use paramers.csv to input fittign parameters.

The Fig4.3 is an example of auto-fitting of weighted mobiliy to calculate PF and ZT under $\kappa_L = 0.44$ at fixed temeprature 750 K and effective mass of 3.2 from Fig4.2b.



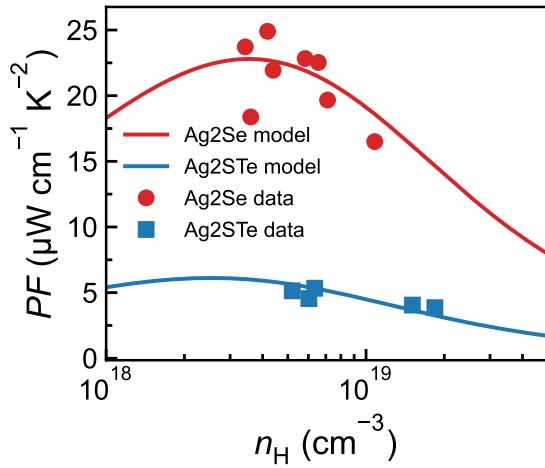
(a) Fitting of PF at 750 K.



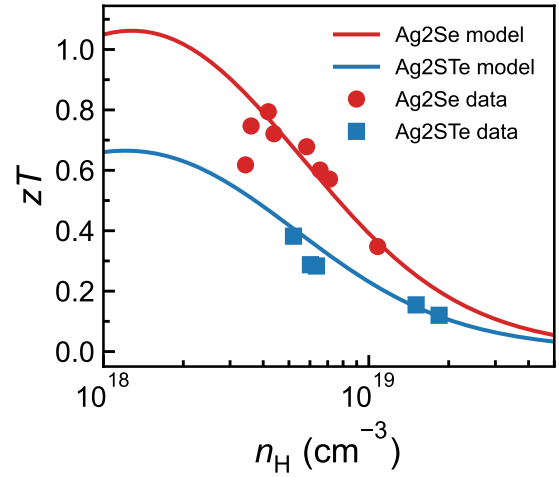
(b) Fitting of zT at 750 K.

Figure 4.3: Performance fitting of Cu2Se.

To fit multiple sets of data, using Listing 4.3 with raw data and parameters, the results can be seen in Fig 4.4.



(a) Fitting of PF at 300 k.



(b) Fitting of zT at 300 K.

Figure 4.4: Multiple sets data fitting of Ag2SSeTe.

Listing 4.3: Command slot: SPB performance fit.

```
python main.py spb performance \
data/demo/spb_fitting/Ag2Se/Ag2Se_series.csv \
--group-by curve_id \
--params data/demo/spb_fitting/Ag2Se/Ag2Se_params.csv \
--output-dir results/spb_fitting/performance_fit \
--properties pf zt \
--formats png pdf \
--legend outside \
--no-show
```

Listing 4.4: Terminal-output slot: performance fit.

```
# TODO: paste key saved-file lines here.
# Expected outputs include:
# results/spb_fitting/performance_fit/performance_summary.json
# results/spb_fitting/performance_fit/performance_curve.csv
# results/spb_fitting/performance_fit/pf_fit.png
# results/spb_fitting/performance_fit/zt_fit.png
```

4.6 Conductivity-Axis Fitting

The conductivity-axis workflow is useful when the experimental series is more naturally interpreted as Seebeck, power factor, or zT versus electrical conductivity. The script fits or uses a weighted mobility prefactor and renders model curves directly on the conductivity axis.

Listing 4.5: Command slot: conductivity-axis SPB fit.

```
python main.py spb conductivity \
  data/demo/spb_fitting/Cu2SSeTe/900K.csv \
  --temperature 900 \
  --x conductivity \
  --y S \
  --pf PF \
  --zt ZT \
  --sigma-unit S/cm \
  --kL 0.3 \
  --output-dir results/spb_fitting/conductivity_fit \
  --properties seebeck pf zt \
  --formats png pdf \
  --legend outside \
  --no-show
```

Listing 4.6: Terminal-output slot: conductivity-axis fit.

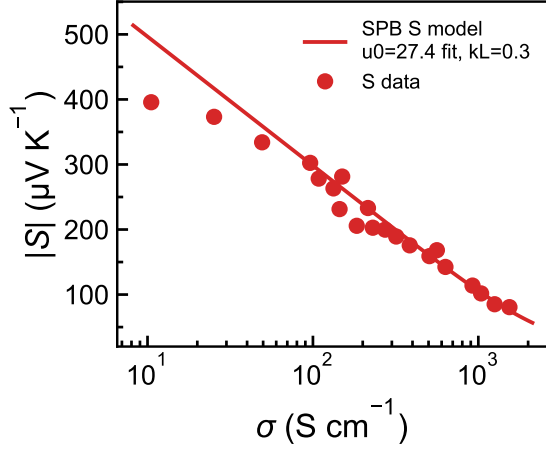
```
# TODO: paste key saved-file lines here.
# Expected outputs include:
# results/spb_fitting/conductivity_fit/conductivity_summary.json
# results/spb_fitting/conductivity_fit/conductivity_curve.csv
# results/spb_fitting/conductivity_fit/conductivity_seebeck_fit.png
# results/spb_fitting/conductivity_fit/conductivity_pf_fit.png
# results/spb_fitting/conductivity_fit/conductivity_zt_fit.png
```

4.7 Interpreting the Outputs

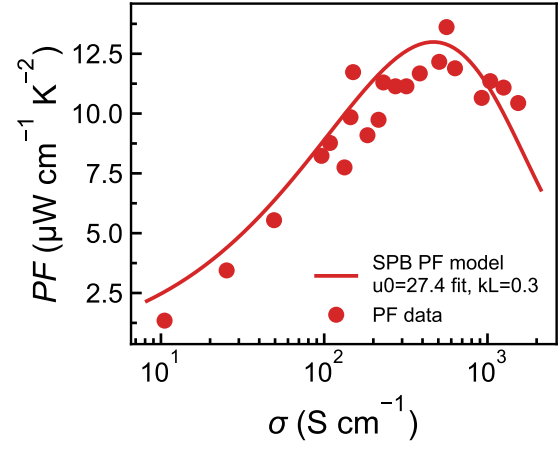
Summary JSON files. The JSON summaries are the best place to check fitted scalar parameters such as effective mass, mobility prefactor, lattice thermal conductivity setting, and fit configuration. They should be archived with any figure used in a report.

Point CSV files. The point files keep the experimental measurements after unit normalization and after derived SPB quantities have been added. These are useful for diagnosing outliers, sign conventions, and incorrect input columns.

Curve CSV files. The curve files store the model line sampled over the plotting range. They make the figure reproducible even if plotting styles change later.



(a) Fitted S .



(b) Fitted PF .

Figure 4.5: Fitted conductivity *v.s.* thermoelectric parameters.

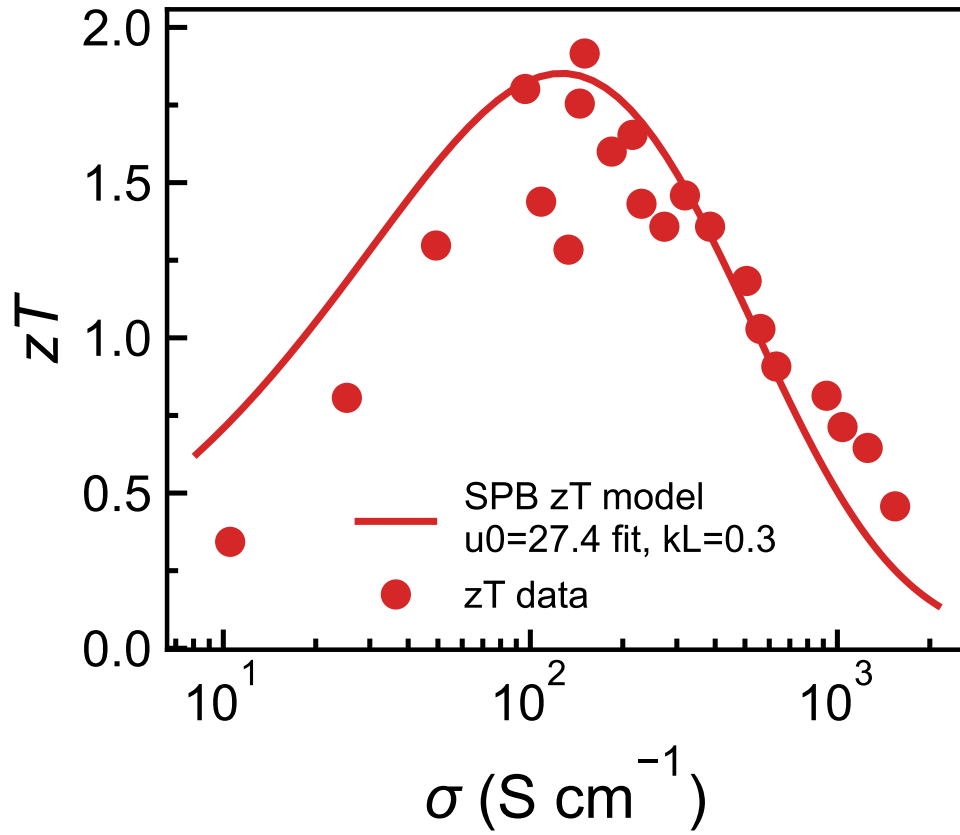


Figure 4.6: Fitted zT value.

Figures. The PNG/PDF outputs are intended for quick inspection and report insertion. Prefer PDF or SVG-like vector outputs when the figure will be edited or placed in a manuscript.

4.8 Quality-Control Checklist

1. Confirm the carrier concentration unit: cm^{-3} and m^{-3} differ by 10^6 .
2. Confirm the Seebeck unit: $\mu\text{V}/\text{K}$ and V/K differ by 10^6 .
3. Confirm whether the Hall factor should be included. The default is to include the acoustic-SPB Hall factor.
4. Avoid mixing temperatures in a single fit unless `--group-by` and `--T-column` or a parameter CSV explicitly separate them.
5. Inspect residuals or point-vs-curve disagreement before interpreting a fitted effective mass as a material constant.
6. Record the exact command, parameter CSV, and output directory for each figure used in a presentation or report.

4.9 Open Slots for Future Expansion

Theory slot. Add a fuller derivation of the SPB transport coefficients, including Lorenz number and the selected scattering exponent.

Validation slot. Add one benchmark table comparing fitted values against a literature SPB example or a manually reproduced calculation.

Lab-data slot. Add a CHY batch example once a complete Hall-containing dataset is selected for publication-style analysis.

Chapter 5

Plotting and Figures

5.1 Introduction

This chapter will introduce methods for standard TE plotting, flexible plotting recipes.

5.2 Fast TE Plotting

Based on the Raw TE Analysis in Chapter 3, the processed TE data are listed in `./data/processed/batch`. The fast plotting can be achieved by `python main.py plot-te`

Listing 5.1: Fast TE plotting.

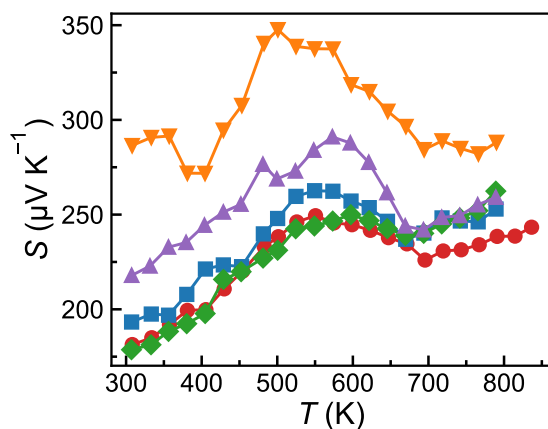
```
python main.py plot_te BATCH_ID SAMPLE_ID \  
--seebeck --conductivity --power_factor \  
--xlim LOW HIGH --ylim LOW HIGH \  
--legend
```

The flexible plotting recipes also include temperature dependent thermoelectric properties.

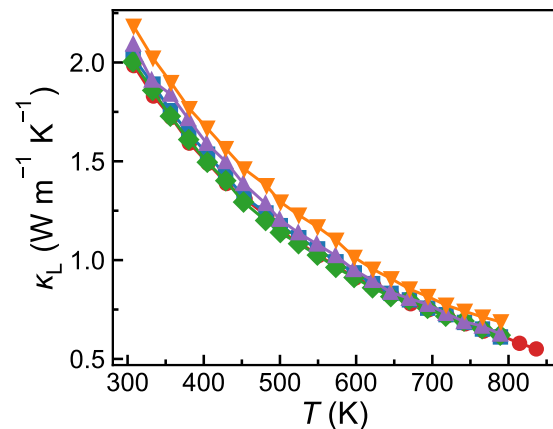
Listing 5.2: TE plotting command slot.

```
python main.py flexible \  
--recipe configs/plot_recipes/thermoelectric/temperature_vs_seebeck.json \  
data/demo/thermoelectric_property/1.csv \  

```



(a) Single batch S .



(b) Single batch κ_L .

Figure 5.1: Fast TE plotting.

5.2.1 TE properties comparison inter batches

It's important to introduce the methods to plot inter-batches comparison of thermoelectric properties. If all data are recorded in `data\processed` with `batch_id`, the fastest way is to run `python plot_te.py`.

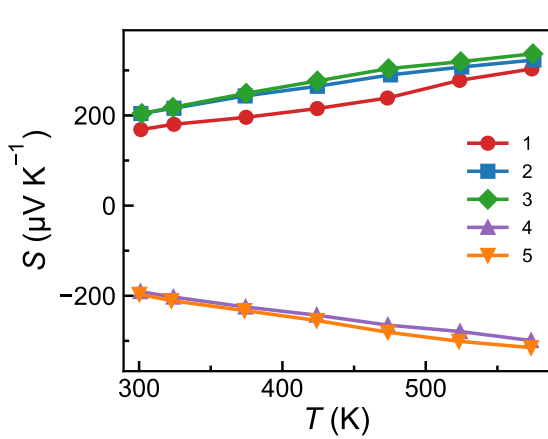
Listing 5.3: Fast inter-batch TE plotting.

```
python main.py plot_te BATCH_ID_1 BATCH_ID_2 BATCH_ID_3
```

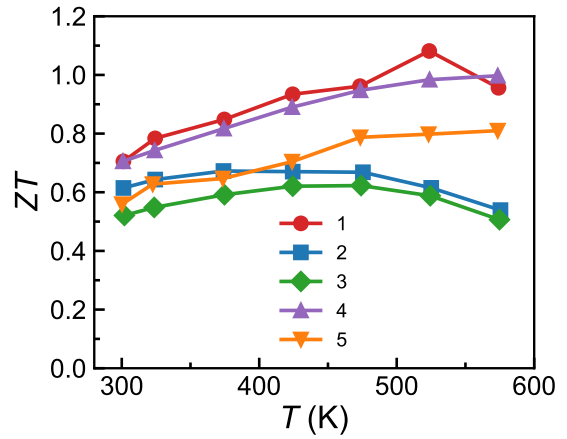
Most generally, comparison with references data, to use `flexible_plot.py` under different recipes is recomanded. The data of each sample should have columns with the title used in the program.

Listing 5.4: Plotting multiple reference data.

```
python main.py flexible \
--recipe configs/plot_recipes/thermoelectric/temperature_vs_seebeck.json \
data/demo/thermoelectric_property/{1,2,3,4,5}.csv \
--x T --y S \
--label 'Sample1' --label '$Cu_{0.2}$' --label '$Cu_{0.4}$' \
--stem seebeck_comparison
```



(a) Single batch S .



(b) Single batch κ_L .

Figure 5.2: Inter batch plotting.

5.3 Planned Content

- Standard single-property and combined TE plots.
- Cross-batch comparison figures.
- Flexible plotting direct CLI mode versus recipe mode.
- Output formats: PNG for quick review, PDF/SVG for manuscripts.
- Axis-limit conventions and legend placement.

/

Chapter 6

XRD Analysis

6.1 Scope

This chapter will document XRD file layout, normalized/stacked plotting, PDF-card overlays, and lattice-parameter fitting.

6.2 Command Slot

Listing 6.1: XRD plotting command slot.

```
# TODO: replace with a real XRD command.  
python main.py plot-xrd CHY-1038 --no-show  
python main.py plot-xrd CHY-1038 --pdf-card CuInTe2_PDF_97_023_8958 --no-show
```

6.3 Planned Content

- Expected XRD raw file naming.
- Normalized versus not-normalized plotting modes.
- PDF-card standard CSV requirements.
- Lattice fitting inputs and outputs.
- Recommended annotations for phase assignment.

Missing figure
Expected file:
outputs/figures/xrd/example.png

Figure 6.1: Figure slot for a stacked XRD plot.

Chapter 7

Assessment and Prediction

7.1 Scope

This chapter will document deterministic batch assessment, AI-assisted interpretation, and local Bayesian prediction for follow-up sample selection.

7.2 Command Slot

Listing 7.1: Assessment and prediction command slot.

```
# TODO: replace with selected batches.  
python main.py assess CHY-1040 CHY-1048  
python main.py bayes --source lab --target ZT_max --top 10
```

7.3 Planned Content

- What features are extracted from processed TE data.
- How deterministic scoring differs from model-assisted assessment.
- How Bayesian prediction ranks candidates.
- How to record evidence sources in the final recommendation.

Result slot. Paste a representative ranked-candidate table and link to the corresponding report under `results/bayesian_predictions/`.

Chapter 8

Reproducible Operations

8.1 End-to-End Protocol

1. Place original instrument files under `data/raw/<batch_id>/`.
2. Run metadata synchronization.
3. Fill missing density, heat-capacity, composition, and notes fields.
4. Run raw TE analysis.
5. Generate TE and XRD figures.
6. Run optional SPB, assessment, and prediction workflows.
7. Archive the exact commands, output paths, and figure files used in any report.

8.2 Command Slot

Listing 8.1: End-to-end workflow command slot.

```
python scripts/sync_lab_metadata.py
python scripts/sync_lab_markdown.py export
# TODO: edit data/lab/lab_metadata.md
python scripts/sync_lab_markdown.py
python main.py analyze CHY-1048
python main.py plot-te CHY-1048 --seebeck --no-show
python main.py plot-xrd CHY-1048 --no-show
python main.py assess CHY-1048
```

8.3 Output Log Slot

Listing 8.2: End-to-end output log slot.

```
# TODO: paste final output paths here.
# Processed CSV:
# TE figure:
# XRD figure:
# Assessment report:
```